

Math 156 Assignment 4

October 25, 2024

```
[14]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import MinMaxScaler
```

```
[49]: # Sigmoid function
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

# Loss function: Binary Cross-Entropy
def cross_entropy_loss(y, t):
    epsilon = 1e-15 # small value to prevent log(0)
    t = np.clip(t, epsilon, 1 - epsilon)
    return -np.mean(t * np.log(y) + (1 - t) * np.log(1 - y))
```

```
[50]: class LogisticRegression:
    def __init__(self, learning_rate, batch_size, max_iters):
        # required hyperparameters
        self.learning_rate = learning_rate
        self.batch_size = batch_size
        self.max_iters = max_iters

    def fit(self, X, t):
        N, D = X.shape
        # initialize weights to 0s
        self.w = np.zeros(D)

        for i in range(self.max_iters):
            # shuffle rows
            indices = np.arange(N)
            np.random.shuffle(indices)
            self.w = np.random.randn(D)
            # loop through the batch size
            for start_idx in range(0, N, self.batch_size):
                end_idx = min(start_idx + self.batch_size, N)
                # obtain the batch indices
```

```

        batch_indices = indices[start_idx:end_idx]
        # feature batch
        X_batch = X[batch_indices]
        # target batch
        t_batch = t[batch_indices]

        y_batch = sigmoid(np.dot(X_batch, self.w))

        # average gradient of each batch
        gradient = np.dot(X_batch.T, (y_batch - t_batch)) / len(t_batch)

        # Update weights
        self.w -= self.learning_rate * gradient

def predict_proba(self, X):
    #probabilities
    return sigmoid(np.dot(X, self.w))

def predict(self, X):
    # class label
    return (self.predict_proba(X) >= 0.5).astype(int)

def accuracy(self, X, t):
    predictions = self.predict(X)
    return np.mean(predictions == t)

def confusion_mat(self, y_true, y_pred):
    TP = np.sum((y_true == 1) & (y_pred == 1))
    TN = np.sum((y_true == 0) & (y_pred == 0))
    FP = np.sum((y_true == 0) & (y_pred == 1))
    FN = np.sum((y_true == 1) & (y_pred == 0))
    return TP, FP, FN, TN

def precision_recall_f1(self, y_true, y_pred):
    TP, FP, FN, TN = self.confusion_mat(y_true, y_pred)

    # Precision
    precision = TP / (TP + FP) if (TP + FP) > 0 else 0

    # Recall
    recall = TP / (TP + FN) if (TP + FN) > 0 else 0

    # F1-Score
    f1 = 2 * (precision * recall) / (precision + recall) if (precision +
↪recall) > 0 else 0

    return precision, recall, f1

```

```

#evaluation function, calculates loss and returns all metrics
def evaluate(self, X, t):
    y_pred = self.predict(X)
    y_prob = self.predict_proba(X)

    loss = cross_entropy_loss(y_prob, t)
    accuracy = self.accuracy(X, t)

    precision, recall, f1 = self.precision_recall_f1(t, y_pred)

    return loss, accuracy, precision, recall, f1

```

0.1 Exercise 4

a) and b) loading in data + train test split

```

[51]: from sklearn.datasets import load_breast_cancer
      from sklearn.model_selection import train_test_split
      from sklearn.preprocessing import StandardScaler

      cancer = load_breast_cancer()

```

```

[52]: X = cancer.data # Features
      y = cancer.target

      #scale data
      scaler = StandardScaler()
      X_scaled = scaler.fit_transform(X)

      #split data into training, validation and test splits
      X_train_val, X_test, y_train_val, y_test = train_test_split(X_scaled, y,
      ↪test_size=0.2, random_state=42, stratify=y)
      X_train, X_val, y_train, y_val = train_test_split(X_train_val, y_train_val,
      ↪test_size=0.25, random_state=42, stratify=y_train_val)

```

0.2 c) size of each class

```

[53]: yc = np.concatenate([y_train, y_val])

      # Report the size of each class
      class_0_count = sum(yc == 0)
      class_1_count = sum(yc == 1)

      print(f"Class 0 count in training + validation set: {class_0_count}")
      print(f"Class 1 count in training + validation set: {class_1_count}")

```

Class 0 count in training + validation set: 170
 Class 1 count in training + validation set: 285

0.3 d)

```
[54]: batch_sizes = [8,16,32]
learning_rates = [0.1,0.01,0.0001,0.000001]
results = list()

for batch_size in batch_sizes:
    for learning_rate in learning_rates:
        #fit model
        model = LogisticRegression(learning_rate, batch_size, max_iters=5000)
        model.fit(X_train, y_train)

        #evaluate loss and results
        train_loss, train_acc, train_prec, train_rec, train_f1 = model.
        evaluate(X_train, y_train)
        val_loss, val_acc, val_prec, val_rec, val_f1 = model.evaluate(X_val,
        y_val)

        results.append([batch_size,learning_rate,train_loss, train_acc,
        train_prec, train_rec, train_f1,val_loss, val_acc, val_prec, val_rec,
        val_f1])

r = pd.DataFrame(results, columns=['Batch Size','Learning_Rate','Train_
Loss','Train Acc','Train Prec','Train Rec', 'Train F1', 'Val Loss', 'Val_
Acc', 'Val Prec', 'Val Rec',
                                'Val F1'])

print(r)
```

```
/var/folders/l1/86nzn1vlj3313vn4h__34jl100000gn/T/ipykernel_25518/3502039715.py:9
: RuntimeWarning: divide by zero encountered in log
return -np.mean(t * np.log(y) + (1 - t) * np.log(1 - y))
```

	Batch Size	Learning_Rate	Train Loss	Train Acc	Train Prec	Train Rec	\
0	8	0.100000	0.226166	0.914956	0.926267	0.939252	
1	8	0.010000	inf	0.258065	0.346457	0.205607	
2	8	0.000100	2.898296	0.413490	0.542683	0.415888	
3	8	0.000001	0.902038	0.521994	0.662420	0.485981	
4	16	0.100000	0.605295	0.812317	0.878788	0.813084	
5	16	0.010000	1.017061	0.583578	0.697802	0.593458	
6	16	0.000100	2.351282	0.419355	0.550000	0.411215	
7	16	0.000001	2.134169	0.322581	0.437956	0.280374	
8	32	0.100000	0.636251	0.759531	0.879310	0.714953	
9	32	0.010000	1.239065	0.680352	0.751196	0.733645	
10	32	0.000100	3.777521	0.134897	0.200000	0.126168	

11	32	0.000001	4.618255	0.331378	0.450000	0.294393
----	----	----------	----------	----------	----------	----------

	Train F1	Val Loss	Val Acc	Val Prec	Val Rec	Val F1
0	0.932715	0.198134	0.921053	0.955882	0.915493	0.935252
1	0.258065	2.760519	0.219298	0.285714	0.169014	0.212389
2	0.470899	3.126591	0.377193	0.500000	0.380282	0.432000
3	0.560647	0.959851	0.535088	0.704545	0.436620	0.539130
4	0.844660	0.356155	0.885965	0.926471	0.887324	0.906475
5	0.641414	1.290469	0.517544	0.621212	0.577465	0.598540
6	0.470588	2.458663	0.412281	0.538462	0.394366	0.455285
7	0.341880	2.075658	0.359649	0.480000	0.338028	0.396694
8	0.788660	0.733192	0.719298	0.819672	0.704225	0.757576
9	0.742317	0.961954	0.640351	0.727273	0.676056	0.700730
10	0.154728	3.306572	0.131579	0.111111	0.056338	0.074766
11	0.355932	4.289751	0.350877	0.466667	0.295775	0.362069

From the results above, it seems that batch size of 8 and learning rate of 0.1 is optimal.

0.4 e)

```
[55]: #fit true model
model = LogisticRegression(learning_rate = 0.1, batch_size = 8, max_iters=5000)
model.fit(X_train, y_train)

# Evaluate on training, validation, and test sets
train_loss, train_acc, train_prec, train_rec, train_f1 = model.
    ↪evaluate(X_train, y_train)
val_loss, val_acc, val_prec, val_rec, val_f1 = model.evaluate(X_val, y_val)
test_loss, test_acc, test_prec, test_rec, test_f1 = model.evaluate(X_test,
    ↪y_test)

print(f"Training set: Loss = {train_loss}, Accuracy = {train_acc}, Precision =
    ↪{train_prec}, Recall = {train_rec}, F1-score = {train_f1}")
print(f"Validation set: Loss = {val_loss}, Accuracy = {val_acc}, Precision =
    ↪{val_prec}, Recall = {val_rec}, F1-score = {val_f1}")
print(f"Test set: Loss = {test_loss}, Accuracy = {test_acc}, Precision =
    ↪{test_prec}, Recall = {test_rec}, F1-score = {test_f1}")
```

```
Training set: Loss = 0.2252349284695633, Accuracy = 0.9002932551319648,
Precision = 0.9545454545454546, Recall = 0.883177570093458, F1-score =
0.9174757281553398
Validation set: Loss = 0.21194328757782116, Accuracy = 0.9035087719298246,
Precision = 0.8947368421052632, Recall = 0.9577464788732394, F1-score =
0.9251700680272109
Test set: Loss = 0.2705207066515404, Accuracy = 0.8947368421052632, Precision =
0.9838709677419355, Recall = 0.8472222222222222, F1-score = 0.9104477611940298
```

0.5 f) Summarization of Findings

The final model that was decided upon used hyperparameter values of a learning rate of 0.1, a batch size of 8 and 5000 max iterations. It was able to generalize fairly well to the test set. The cross-entropy loss was calculated to be around 0.271, the accuracy 0.895, precision 0.984, the recall 0.8472 and the F1-score was 0.9104. The relatively low recall means that it did particularly poorly when correctly identifying positive values.